

Communication Diagram Collaboration 정의

1. 문서 목적과 범위

이 문서는 프로젝트의 세 Event-Action Table과 BCE 클래스 초안을 바탕으로 다음 두 가지만 정의한다.

1. Communication Diagram을 어떤 collaboration 단위로 나눌 것인가.
2. 각 collaboration에서 참여 클래스가 어떤 역할을 맡는가.

이 문서는 메시지 번호, 호출 순서, 상세 guard, 상태 전이, 재시도 횟수 같은 동작 흐름을 설계하지 않는다. 그 내용은 별도로 구현할 State Machine(STM)과 Event-Action Table의 책임이다.

분석 기준 문서는 다음과 같다.

- [UI Event-Action Table](#)
- [Trading Logic Event-Action Table](#)
- [4H REGIME Event-Action Table](#)
- [UI Behavior](#)
- [UI Rule](#)
- [4H REGIME 설계](#)
- [하단 볼린저밴드 전략 기술서](#)
- [메인 화면 조회 Use Case](#)
- [거래 내역 상세 조회 Use Case](#)

현재 저장소에는 실행 가능한 구현 소스가 없고 Visual Paradigm 프로젝트에도 Class Diagram과 Communication Diagram이 아직 없다. 따라서 아래 내용은 코드 역공학 결과가 아니라 기존 설계 문서를 정리한 설계 초안이다.

2. Collaboration과 STM의 책임 분리

2.1 기본 원칙

Communication Diagram은 객체 간 협력 관계를 보여주고, STM은 상태 변화 규칙을 구현한다.

구분	책임
Communication collaboration	하나의 사용자 목적 또는 시스템 책임에 필요한 Actor, Boundary, Control, Entity를 선정하고 각 역할을 정의한다.
State Machine	현재 상태, 수신 가능한 event, guard 판정, 다음 상태, entry/exit action을 정의한다.
Controller	외부 event와 필요한 context를 STM에 전달하고, STM이 선택한 action을 Gateway·Entity·UI를 통해 수행한다.
Boundary	사용자 또는 외부 시스템과 입출력한다. 상태 전이를 직접 결정하지 않는다.
Entity	업무 데이터와 계산 결과를 보존한다. 다음 상태를 직접 선택하지 않는다.

Communication Diagram에 STM 객체를 참여시킬 수는 있지만, STM 내부의 전이들을 메시지 흐름으로 다시 풀어 그리지 않는다. 해당 위치에는 “상세 상태 전이는 관련 State Diagram 참조”라는 note를 붙인다.

2.2 별도 구현할 STM

세 Event-Action Table에 대응해 다음 세 개의 최상위 STM 구현을 두는 것을 권장한다.

STM 구현 후보	소유 Controller	기준 문서	책임
UIStateMachine	UIStateController	UI Event-Action Table	화면, 팝업, 탭, 필터, 차트 조작, 시작·중지·종료 UI 상태를 관리한다.
TradingStateMachine	TradingCoordinator	Trading Logic Event-Action Table	자동매매 생명주기, 하단 BB 이벤트, Case B/C 병렬 region, position owner, 진입·보유·청산 상태를 관리한다.
RegimeStateMachine	RegimeController	4H REGIME Event-Action Table	4H 입력에 따른 추천 REGIME 판정 상태를 관리한다.

TradingStateMachine의 Case B signal, Case C signal, Case B position, Case C position region을 실제 코드에서 하위 STM 클래스로 분리할 수 있다. 다만 Communication Diagram에서는 하나의 tradingSTM:TradingStateMachine 역할 객체로 표현해도 충분하다.

STM은 Binance API, WebSocket, UI, 파일 시스템을 직접 호출하지 않는다. STM은 transition 결과와 수행할 action의 종류만 반환하고 실제 I/O는 소유 Controller가 담당한다.

2.3 Controller와 STM의 관계

- UIStateController는 UI event를 해석하는 facade이며 UI 상태 자체는 UIStateMachine이 결정한다.
- TradingCoordinator는 거래 업무의 orchestrator이며 거래 상태 자체는 TradingStateMachine이 결정한다.
- InvestmentLogicController는 지표와 가격으로 전략 조건을 계산해 TradingStateMachine이 사용할 판정 context를 만든다.
- RegimeController는 4H 입력 snapshot을 준비하고 RegimeStateMachine의 판정 결과를 RegimeResult로 변환한다.
- Controller가 STM guard를 중복 구현하거나 STM이 Controller의 I/O 책임을 가져서는 안 된다.

3. 클래스별 기본 역할

3.1 Boundary

클래스	역할
MainScreenUI	추천/선택 REGIME, 계좌 요약, 투자 로직 상태, 분할 비율을 표시하고 사용자 입력을 전달한다.
ChartPanelUI	candle, 지표, active state, 주기, 전체 화면, drawing을 표시한다.
RecentOrderUI	최근 주문 요청이 아니라 최근 체결 결과를 표시한다. 의미를 명확히 하려면 RecentTradeUI로 변경할 수 있다.
TradeHistoryUI	거래 내역 상세, 성과, 기간/side 필터, CSV 내보내기 진입점을 표시한다.

클래스	역할
PopupUI	시작, REGIME 변경, 중지, 강제 매도, 연결 오류, CSV 결과 등의 modal UI를 담당한다. 역할 객체는 <code>startPopup</code> , <code>stopPopup</code> , <code>csvPopup</code> 처럼 구분한다.
BinanceAPIGateway	Binance REST API와의 연결 확인, 계좌·candle 조회, 주문 제출·조회·취소를 캡슐화한다.
BinanceWebSocketGateway	market/kline/user-data stream 연결과 수신 event 정규화를 담당한다.

다음 Boundary는 기존 초안에 없지만 collaboration을 명확히 나누려면 추가하는 편이 좋다.

추가 클래스	역할
AppShellUI 또는 UpperStatusBarUI	API 상태, 자동매매 시작·중지, 프로그램 종료처럼 Main/Trading Details와 독립적인 전역 UI를 담당한다. 추가하지 않으면 <code>MainScreenUI</code> 가 이 역할을 함께 맡는다.
TradeHistoryRepository	내부 거래 이력의 load/save를 담당한다. 화면이나 CSV export 파일과 분리한다.
CSVFileGateway	저장 위치 선택, 파일명 검증 결과 반영, CSV 파일 쓰기와 I/O 오류를 격리한다.

3.2 Control

클래스	역할
UIStateController	모든 UI Boundary의 단일 제어 진입점이다. <code>UIStateMachine</code> 을 실행하고 선택된 UI action을 각 Boundary에 반영한다.
TradingCoordinator	자동매매 시작·중지, 단일 position owner, 주문 의도, 체결 결과 반영, 장애 복구를 조정한다. 상태 결정은 <code>TradingStateMachine</code> 에 위임한다.
RegimeController	동일 시점의 4H 입력을 준비하고 <code>RegimeStateMachine</code> 을 실행한다. 추천값을 사용자 선택값에 자동 적용하지 않는다.
TradeHistoryController	거래 저장, 조회, 필터, 성과 계산, repository와 CSV export를 조정한다.
UIStateMachine	UI Event-Action Table을 구현한다.
TradingStateMachine	Trading Logic Event-Action Table을 구현한다.
RegimeStateMachine	4H REGIME Event-Action Table을 구현한다.

다음 Control은 구현 시 역할 분리를 위해 권장한다.

추가 클래스	역할
MarketDataController	REST candle과 WebSocket 실시간 데이터를 정규화하고 동일 시점의 market/indicator snapshot을 제공한다.

3.3 Entity와 Value Object

클래스	필요한 데이터와 역할
Account	자산 잔액, ETH 보유량, 현재가, 평가금액을 보존한다.
Position	symbol, owner 전략, 수량, 평균 진입가, 진입 시각, 상태, 청산 사유를 보존한다.
Order	client/exchange order ID, side, 유형, 요청 수량·금액, 체결 수량, 상태, 실패 사유를 보존한다.
Trade	실제 fill 단위의 체결 시각, 체결가, 수량, 금액, 수수료, 전략, 손익, 청산 사유를 보존한다.
TradeHistory	Trade 목록을 보유하고 최근/기간/side 조회의 원천이 된다.
Performance	당일 수익률, 누적 수익률, 실현 손익, 수수료를 제공한다. 필요하다면 평균 매도 수익률, 총수익, 승/패도 파생한다.
StrategySetup	symbol, quote asset, 사용자 선택 REGIME 을 보존한다.

다음 데이터 객체도 collaboration 간 결합을 낮추는 데 필요하다.

추가 클래스	역할
MarketSnapshot	현재가와 timeframe별 candle을 동일 평가 시점 기준으로 묶는다.
IndicatorSnapshot	%B, BB, BBW, EMA slope, CCI, timeframe, 확정봉 여부를 묶는다.
TradingContext	lowerEventId , position owner, Case B/C 판정에 필요한 runtime 값, 분할 매수·매도 비율을 보존한다. 활성 상태와 전이 권한은 TradingStateMachine 에 둔다.
ConnectionStatus	REST, market stream, user-data stream, trading-ready 상태를 구분한다.
TradeHistoryQuery	기간과 side 필터를 하나의 조회 조건으로 묶는다.
CSVExportOptions	저장 위치, 기간, 시작·종료일, 파일명을 묶는다.

Performance에서 입출금 보정 수익률을 실제 계산하려면 **AccountSnapshot**과 **CashFlowHistory**가 추가로 필요하다. 데이터 출처가 확정되지 않았다면 해당 값은 계산 불가 상태로 표시해야 한다.

3.4 외부 Actor

Actor	역할
user:User	화면 조작과 확인·취소를 수행한다.
binanceRest:Binance REST API	계좌, candle, 주문 API를 제공한다.
binanceStream:Binance WebSocket	실시간 market 및 체결 event를 제공한다.
fileSystem:Local File System	내부 거래 이력과 CSV 파일을 저장한다.
clock:System Clock	TradingScheduler 에 시간 기준을 제공한다.

4. 필요한 Core Collaboration

COL-01 애플리케이션 초기화와 거래소 연결

- 목적: 메인 화면을 열고 거래소 연결 상태, 계좌, 초기 candle, 거래 이력을 사용할 수 있게 한다.
- Boundary 역할: **AppShellUI/MainScreenUI**는 초기 상태를 표시하고, 두 Binance Gateway는 외부 데이터를 제공한다.
- Control/STM 역할: **UIStateController**와 **UIStateMachine**은 초기 UI 상태를 정한다. **TradingCoordinator**는 Gateway와 초기 업무 데이터를 조정한다. **MarketDataController**는 초기 market snapshot을 만든다.
- Entity 역할: **Account, ConnectionStatus, TradeHistory, StrategySetup**이 초기 표시 데이터가 된다.
- STM 참조: UI 초기 상태만 **UIStateMachine**에서 처리한다. 연결 절차 자체는 STM 전이로 만들 필요가 없다.

COL-02 실시간 시장 데이터 갱신

- 목적: WebSocket 데이터를 전략 판단, REGIME 판단, 차트, 계좌 평가에 공통으로 공급한다.
- Boundary 역할: **BinanceWebSocketGateway**는 외부 event를 내부 형식으로 정규화한다. **ChartPanelUI**와 **MainScreenUI**는 결과만 표시한다.
- Control/STM 역할: **MarketDataController**는 **MarketSnapshot**과 **IndicatorSnapshot**을 만든다. **TradingCoordinator**는 필요한 Controller에 배포한다. **InvestmentLogicController**와 **RegimeController**는 각 목적에 맞는 값을 계산한다.
- Entity 역할: **MarketSnapshot, IndicatorSnapshot, Account**가 최신 데이터를 보존한다.
- STM 참조: market event가 상태 event로 필요한 경우에만 **TradingStateMachine** 또는 **RegimeStateMachine**에 전달한다.

COL-03 4H REGIME 추천

- 목적: 4H 기준으로 추천 REGIME을 계산하고 화면에 표시한다.
- Boundary 역할: **MainScreenUI**는 추천값과 판정 지표를 표시한다.
- Control/STM 역할: **RegimeController**는 입력 snapshot과 판정 context를 준비하고 **RegimeStateMachine**을 실행한다. **UIStateController**는 표시 상태만 갱신한다.
- Entity 역할: **IndicatorSnapshot**은 판정 입력이고 **RegimeResult**는 추천 결과다.
- STM 참조: EA-001~008, EA-101~105의 guard와 다음 상태는 **RegimeStateMachine**에서만 구현한다.

COL-04 사용자 REGIME 선택과 변경

- 목적: 추천값과 별개로 사용자가 실제 적용할 REGIME을 선택하거나 변경한다.
- Boundary 역할: **MainScreenUI**는 선택 입력과 선택 상태를 표시하고 **PopupUI**는 필요한 확인·취소 UI를 담당한다.
- Control/STM 역할: **UIStateController**와 **UIStateMachine**은 팝업과 선택 UI 상태를 관리한다. **TradingCoordinator**는 확정된 선택을 향후 거래 판단에 적용한다.
- Entity 역할: **StrategySetup.selectedRegime**이 사용자 선택값을 보존하고 **RegimeResult.recommendedType**은 추천값을 보존한다.
- STM 참조: UI 확인·취소 전이는 **UIStateMachine**, 실행 중 전략 적용 상태는 **TradingStateMachine**의 책임이다.

COL-05 자동매매 시작

- 목적: 연결 상태와 설정을 확인하고 자동매매를 활성화한다.
- Boundary 역할: **AppShellUI/MainScreenUI**는 시작 입력과 실행 상태를 표시하고 **PopupUI**는 확인 또는 오류 안내를 담당한다.

- Control/STM 역할: **UIStateController**는 UI event를 처리하고, **TradingCoordinator**는 시작 가능 context를 구성해 **TradingStateMachine**에 전달한다. **InvestmentLogicController**는 선택된 전략 조건 계산을 준비한다.
- Entity 역할: **ConnectionStatus**, **StrategySetup**, **TradingContext**, **Position**이 시작 context를 제공한다.
- STM 참조: **LOGIC_ENABLED**, **LOWER_TOUCH_WATCH** 같은 상태 전이는 **TradingStateMachine**에서 구현한다.

COL-06 하단 BB 기반 진입 판단

- 목적: 하단 BB 접촉 이후 Case B와 Case C를 병렬로 평가하되 실제 position owner는 하나만 선택한다.
- Boundary 역할: 이 collaboration에는 일반적으로 UI Boundary가 필요하지 않다. 필요하면 **MainScreenUI**와 **ChartPanelUI**가 현재 active logic state만 표시한다.
- Control/STM 역할: **InvestmentLogicController**는 Case B/C 조건을 계산한다. **TradingStateMachine**은 두 region의 상태와 단일 owner 선택을 결정한다. **TradingCoordinator**는 선택된 action을 주문 collaboration에 연결한다. **TradingScheduler**는 시간 event만 제공한다.
- Entity 역할: **IndicatorSnapshot**, **Position**, **TradingContext**, **StrategySetup**이 판정 context를 제공한다.
- STM 참조: G, B, C, O 영역의 모든 상태-guard-병렬 종료 조건은 **TradingStateMachine**에서 구현한다.

COL-07 주문 실행과 체결 반영

- 목적: Case B/C 진입, 전략 청산, 사용자 강제 매도가 동일한 주문 처리 책임을 재사용하게 한다.
- Boundary 역할: **BinanceAPIGateway**는 주문 제출·회소·취소를 담당하고 **BinanceWebSocketGateway**는 체결 event를 전달한다. **RecentOrderUI**와 관련 화면은 처리 결과만 표시한다.
- Control/STM 역할: **TradingCoordinator**가 주문 의도와 체결 결과를 조정한다. **TradeHistoryController**가 실제 fill을 기록하고 성과를 갱신한다. **TradingStateMachine**은 주문 결과 event에 따른 거래 상태만 결정한다.
- Entity 역할: **Order**는 주문 상태, **Trade**는 fill, **Position**은 보유 상태, **Account**는 잔액, **TradeHistory**와 **Performance**는 이력과 성과를 보존한다.
- STM 참조: REST 주문 접수와 실제 체결을 구분하되 상세 retry-partial fill 상태는 **TradingStateMachine** 또는 별도 주문 STM에서 정의한다.

COL-08 보유 포지션 감시와 청산

- 목적: Case B 또는 Case C가 소유한 포지션의 청산 조건을 평가하고 청산 action을 주문 collaboration에 연결한다.
- Boundary 역할: **MainScreenUI**와 **ChartPanelUI**는 포지션과 active state를 표시한다.
- Control/STM 역할: **InvestmentLogicController**는 청산 조건을 계산한다. **TradingScheduler**는 지속 시간과 보유 시간 event를 제공한다. **TradingStateMachine**은 Case B/Case C의 보유-trailing-청산 상태를 결정한다. **TradingCoordinator**는 청산 주문을 조정한다.
- Entity 역할: **Position**, **IndicatorSnapshot**, **TradingContext**가 청산 판정의 context가 된다.
- STM 참조: PB와 PC 영역의 상태 및 guard는 **TradingStateMachine**에 남기고 Communication Diagram에서는 Case B/C의 내부 전이를 펼치지 않는다.

COL-09 자동매매 중지와 강제 매도

- 목적: 무포지션 중지와 포지션 보유 중 강제 매도 후 중지를 구분한다.

- Boundary 역할: **AppShellUI/MainScreenUI**는 중지 입력과 상태를 표시하고 **PopupUI**는 확인·취소·실패 안내를 담당한다.
- Control/STM 역할: **UIStateController**와 **UIStateMachine**은 팝업 상태를 관리한다. **TradingCoordinator**와 **TradingStateMachine**은 신규 진입 차단, 필요 시 강제 매도, 최종 중지 상태를 담당한다.
- Entity 역할: **Position, Order, TradingContext**가 중지 유형을 결정하는 context가 된다.
- STM 참조: U2와 G-04~05의 전이는 각각 UI/Trading STM에서 구현한다. 강제 매도 자체는 **COL-07**을 재사용한다.

COL-10 연결 장애와 복구

- 목적: REST, market stream, user-data stream 장애를 구분해 화면과 자동매매 가능 상태에 반영한다.
- Boundary 역할: 두 Binance Gateway가 채널별 연결 상태를 제공하고 **PopupUI/AppShellUI**가 사용자에게 장애를 표시한다.
- Control/STM 역할: **TradingCoordinator**는 신규 진입 차단과 거래소 상태 재동기화를 담당한다. **UIStateController**는 offline UI를 관리한다. STM은 장애 event를 받아 허용된 상태 전이만 결정한다.
- Entity 역할: **ConnectionStatus, Order, Position, Account**가 복구 context가 된다.
- STM 참조: 장애 후 자동 재시작 여부와 open order 복구 정책은 **TradingStateMachine** 명세에서 별도로 결정한다.

COL-11 거래 내역 상세와 성과 조회

- 목적: 거래 목록, 기간/side 필터, 계좌 요약, 성과를 상세 화면에 제공한다.
- Boundary 역할: **RecentOrderUI**는 상세 화면 진입점을 제공하고 **TradeHistoryUI**는 조회 결과와 필터를 표시한다.
- Control/STM 역할: **UIStateController/UIStateMachine**은 Main과 Trading Details 화면 상태 및 필터 UI를 관리한다. **TradeHistoryController**는 결합된 조회 조건과 성과 계산을 담당한다.
- Entity 역할: **TradeHistoryQuery, TradeHistory, Performance, Account**가 조회 입력과 결과가 된다.
- STM 참조: 화면·필터 상태만 **UIStateMachine**에 두고 수익률 계산은 **Performance/TradeHistoryController**의 업무 로직으로 둔다.

COL-12 CSV 내보내기

- 목적: 선택한 조건의 거래 내역을 사용자가 지정한 파일로 내보낸다.
- Boundary 역할: **TradeHistoryUI**는 내보내기 진입점을 제공하고 **PopupUI**는 옵션과 결과를 표시한다. **CSVFileGateway**는 파일 시스템 I/O를 담당한다.
- Control/STM 역할: **UIStateController/UIStateMachine**은 팝업과 validation 표시 상태를 관리한다. **TradeHistoryController**는 export 대상 거래를 조회하고 파일 생성을 조정한다.
- Entity 역할: **CSVExportOptions**와 **TradeHistory**가 입력과 데이터 원천이 된다.
- STM 참조: TD4와 CR1~3의 UI 상태는 **UIStateMachine**에서 구현하고 실제 파일 쓰기는 STM 밖에서 수행한다.

COL-13 프로그램 종료

- 목적: 포지션 유무에 따라 일반 종료 또는 강제 매도 후 종료를 수행한다.
- Boundary 역할: **AppShellUI**와 **PopupUI**가 종료 입력, 확인·취소, 실패 상태를 표시한다. Binance Gateway와 Repository는 외부 자원 정리를 담당한다.

- Control/STM 역할: **UIStateController/UIStateMachine**은 종료 UI 상태를 관리한다.
TradingCoordinator/TradingStateMachine은 신규 진입 차단, 필요 시 강제 매도, 거래 로직 종료를 담당한다. **TradeHistoryController**는 저장 데이터 마감을 담당한다.
- Entity 역할: **Position, Order, TradeHistory, TradingContext**가 종료 context가 된다.
- STM 참조: ES3 전이는 **UIStateMachine**, 거래 종료 상태는 **TradingStateMachine**에서 구현한다. 강제 매도는 **COL-07**을 재사용한다.

5. 보조 Collaboration

ID	Collaboration	참여 클래스와 역할	STM 참조
COL-S1	차트 주기·지표·전체화면·drawing	ChartPanelUI 가 조작과 표시를 담당하고, UIStateController 가 표시 설정을 관리하며, MarketDataController/BinanceAPIGateway 가 candle을 제공한다.	DC1~6, IP1~3은 UIStateMachine 에서 구현한다.
COL-S2	분할 매수·매도 비율 변경	MainScreenUI 가 입력·표시하고, UIStateController 가 UI 상태를 관리하며, TradingCoordinator 가 검증된 값을 StrategySetup 에 반영한다.	SI/SO UI 상태는 UIStateMachine ; 주문 적용 시점은 거래 정책으로 분리한다.
COL-S3	최근 체결·실시간 지표 탭 전환	RecentOrderUI 가 두 표시 모드를 제공하고, UIStateController 가 탭 상태를 관리하며, TradeHistoryController 와 InvestmentLogicController 가 각 표시 데이터를 제공한다.	M4는 UIStateMachine 에서 구현한다.

6. Event-Action 추적성

Event-Action 범위	담당 collaboration	구현 STM
UI U1, ES2-01	COL-01, COL-10	UIStateMachine
UI R1, R3	COL-02, COL-03	UIStateMachine , RegimeStateMachine
UI R2	COL-04	UIStateMachine , TradingStateMachine
UI U3	COL-05	UIStateMachine , TradingStateMachine
UI U2	COL-09	UIStateMachine , TradingStateMachine
UI DC1~6, IP1~3	COL-S1	UIStateMachine
UI ES2-02~03, DI1~2, D1~4, TD2~3	COL-02, COL-07, COL-11	UIStateMachine
UI M4	COL-S3, COL-07	UIStateMachine

Event-Action 범위	담당 collaboration	구현 STM
UI SI, SO	COL-S2	UIStateMachine
UI TD4, CR1~3	COL-12	UIStateMachine
UI ES3	COL-13	UIStateMachine, TradingStateMachine
Trading G, B, C, O	COL-05, COL-06, COL-07, COL-09	TradingStateMachine
Trading PB, PC	COL-07, COL-08	TradingStateMachine
REGIME EA-001~008, EA-101~105	COL-03	RegimeStateMachine

7. STM 명세에서 별도로 해결할 사항

아래 항목은 collaboration 책임이 아니라 STM의 event·guard·transition 명세 문제다. Communication Diagram에서 임의로 해결하지 않는다.

이슈	담당 설계 산출물
EA-007에서 <code>ema9Slope <= -0.30</code> 이지만 <code>LH+LL</code> 이 아닌 경우가 비어 있다.	4H REGIME State Diagram/Event-Action Table
Trading Event-Action의 <code>C-12</code> ID가 중복된다.	Trading State Diagram/Event-Action Table
<code>C-09</code> 가 발생시키는 event와 수신 상태가 맞지 않는다.	Case C 하위 STM
Case C 청산 실패 뒤 복귀 상태가 기존 보유 상태와 trailing 상태를 구분하지 않는다.	Case C position 하위 STM
PB 청산 조건이 동시에 만족할 때 우선순위가 완전히 확정되지 않았다.	Case B position 하위 STM
UI 문서마다 REGIME 변경 확인 팝업 정책이 다르다.	UI State Diagram/Event-Action Table
강제 매도 실패 시 즉시 실패 처리와 3초 간격 4회 retry가 충돌한다.	UI STM과 Trading STM의 interface contract
연결 복구 후 자동 재시작 여부와 미체결 주문 복구 정책이 없다.	Trading State Diagram 및 장애 복구 명세
<code>regime_design.md</code> 는 UPBIT를 언급하지만 BCE와 Gateway는 Binance이고 UI는 KRW/ETH를 표시한다.	시스템 경계와 거래 symbol 설정 명세

8. Communication Diagram 작성 기준

각 diagram은 다음 내용만 포함하면 된다.

- collaboration에 해당하는 Actor와 BCE 역할 객체

- 객체 사이에 협력이 가능함을 나타내는 link
- 각 객체의 책임을 설명하는 note
- 관련 Event-Action 범위와 “상세 전이는 STM 참조” note

다음 내용은 이 단계에서 포함하지 않는다.

- 전체 메시지 교환 순서
- STM 내부 event의 연속 호출
- guard 식의 상세 전개
- retry/backoff 알고리즘
- 상태 진입·종료 action의 구현 순서

우선 COL-01~COL-13을 Core Communication Diagram 후보로 사용하고, UI 세부 동작이 필요할 때만 COL-S1~COL-S3을 별도 diagram으로 작성한다.

9. 완료 체크리스트

- ☐ 각 collaboration이 하나의 사용자 목적 또는 시스템 책임만 가진다.
- ☐ 모든 Boundary는 표시·입출력 역할만 가진다.
- ☐ Controller와 STM의 책임이 분리되어 있다.
- ☐ 상태, event, guard, 다음 상태는 관련 STM에 남아 있다.
- ☐ InvestmentLogicController가 주문 제출이나 상태 전이를 직접 수행하지 않는다.
- ☐ TradingStateMachine이 Binance Gateway나 UI를 직접 호출하지 않는다.
- ☐ 추천 REGIME과 사용자 선택 REGIME이 서로 다른 Entity 속성으로 표현된다.
- ☐ 주문 요청인 Order와 실제 체결인 Trade가 구분된다.
- ☐ 각 Event-Action 범위가 최소 하나의 collaboration과 STM에 매핑된다.